

Instruction Leakage and Optimizations

Homework 4

Part 0: Leakage Mechanisms

1. As the pipeline is synchronized by the clock, all instructions take a constant time to go through it, so the execution time can not differentiate the instruction types, this limits the information leakage. Yet, branch mispredictions and instructions dependencies can cause stalls, which are extra ticks that can be observed and give information about the program's structure. Also, branches can cause different blocks to execute, that are possibly different light so differentiable by the number of clock tick needed to finish the execution.
2. In a uniprocessor environment, the spy program is scheduled by the OS and runs sequentially in between the targeted program. Flush+Reload and Prime+Probe can be performed, but the flush and reload will be harder to schedule: we want the Flush/Prime before the part of the target that we are interested in and Reload/Probe after, so that necessitates a good knowledge about the OS scheduling, which is often obscure. In addition, it won't be possible to arbitrary fine grain our result because the smallest time between flush and reload will be the time attributed to the target by the OS. The result is a more noisy inference about the target's execution. Any other timing related attack has the same constraints.

Part 1: Analyzing Leaked Information

1. My analyze-program.sh is leveraging the varying tick number, that depends on what branch of the program is taken (for example, if the `fort if` statement is verified, the number of ticks to execute the program is 7296055000). This side channel is influenced by the properties of the arithmetic module that in some processors is using a different number of ticks to multiply two register, divide one by the other, or multiply one by 0. In the case of a pipelined processor (with a constant execution time for instructions), it could be interpreted as the effect of predicting that the first branch will be taken, that in case of super secret data is correctly predicted and doesn't loose any cycles, while in case of not super secret data, the wrongly predicted instructions have to be flushed and the right ones have to be retrieved, causing a higher number of clock ticks to finish the execution.
2. The configuration uses `system.cpu = RiscvO3CPU1952y()`. This CPU is using an out-of-order execution (O3). So execution timing depends on instruction types and dependencies and branch predictions and can be observed by a spy. This allows timing side-channel attacks. With `system.membus = SystemXBar()`, the memory bus is shared and allows a spy program to observe delays in memory accesses. On the other hand, the configuration uses `system.cpu.workload = process`, so one process at a time will be running at a time, so the spy and the target won't be run at the same time. It would be challenging to set up the spy, as described in [part 0.2](#) in a uniprocessor. Also here this configuration doesn't have a cache, removing the possibility of attacks such as Flush+Reload and Prime+Probe.

3. In an out-of-order multi-processor with a shared cache, we could leak the number of instructions of memory reads and write (`system.cpu.statIssuedInstType_0::MemRead` or `system.cpu.statIssuedInstType_0::MemWrite`). More realistically we could infer the relative proportion of them. In the processor using out-of-order execution, these instructions take a different time for each execution (contrary to a synchronized pipeline), and can reveal what type of instructions are processed. In a multi-processor it is easier to time other processes because we can issue spy instruction whenever we want (we don't depend on the scheduling of the OS). The shared cache allows to retrieve the timings with Flush+Reload and Prime+Probe. Knowing what kind of instructions are processed (memory or arithmetic) can allow to track the targeted program workload type (cryptographic computations, deep learning, or simulations?), or the phase of the execution (loading the data, performing the computations?).

Part 2: Hiding Leaked Information

1. The strategy to hide any information in the 1952 CPU is to make the multiplication and the division take the same time, meaning take the longest time across the two instructions (the time necessary to make a division). And we pipeline the instructions making them indistinguishable by timing. We test on Leaky prog, and the execution should take the same amount of time in case of the argument being supersecretdata (a lot of divisions) compared to notsosecretdata (a lot of multiplications).



We observe that our method effectively allowed to hide what program is being executed; whether multiplications or divisions are being performed don't impact anymore the execution time on a given processor.

Further, running the experiments at a very low processor speed (10MHz) reveals that the O3 CPU baseline has better performance. This stems from the difference in the IntMultDiv functional unit.

<pre>class IntMultDiv(FUDesc): opList = [OpDesc(opClass="IntMult", opLat=3), OpDesc(opClass="IntDiv", opLat=20, pipelined=False),]</pre>	<pre>class IntMultDivUnit(FUDesc): opList = [OpDesc(opClass="IntMult", opLat=250, pipelined=True), OpDesc(opClass="IntDiv", opLat=250, pipelined=True),]</pre>
O3 CPU	O3 1952y CPU

The O3 CPU uses a different approach to hiding execution latencies that don't require our pipelined 250 full clock cycles ($250/1\text{MHz} = 25\text{ }\mu\text{s}$ per multiplication or division, vs $0.3\text{ }\mu\text{s}$ for the base CPU's multiplication, $2\text{ }\mu\text{s}$ per division).

The only two div/mult units are then used for much longer per instruction by the 1952y CPU. My hypothesis is that the waiting times are then much longer in the ROB and issue queues resulting in a much lower throughput.

2. In order to avoid leaking information in leaky-prog, a possible solution is to compute both the multiplication and division and wait until both are finished before using only the desired result. In terms of microarchitecture, this could be implemented in parallel, resulting in an execution time taking $\max\{t_{\text{mult}}, t_{\text{div}}\}$. Another approach is to in code successively compute the multiplication and the division, but this would reduce the effective parallelization and result in a longer execution.

The code version is simpler to implement on top of existing hardware, but slower. The micro-architecture version is faster but more costly in terms of silicon and energy.

Part 3: Processor Optimization

We want to keep coherent with the RV64M standard extension so we keep all fields the same as the multiplication and differentiate with a funct7 field 0x2 instead of 0x1.

0x1	rs2	rs1	0x0	rd	0x3B	MULW
0x2	rs2	rs1	0x0	rd	0x3B	ZMUL

Part 4: Exposing *zmul* to Software

1. By adding the zmul instruction to the RISC-V ISA, we have essentially implemented a lightweight RISC-V extension. We had to change the hardware in the decode unit and the execution, and would need a new compiler. It is a complex operation to add a new instruction, so it has to be justified by a substantial gain, in performance, security or a combination of both. To become a standard ISA extension, the extension has to be useful for a broad range of

applications. Otherwise it has to stay a proprietary design when it is too specialized, instead of taking expensive space in the broadly distributed RISC-V.

2. Side channels could be categorized in microscopic, at the bit/instruction level or macroscopic, at the program level. If we were to try to track precisely what individual instruction is being executed, we would rather use a tool like Ripes, to track in a wire if the execution being executed has say the FUNCT3 001 or 101. When we're looking more broadly to a full program execution, like leaky-program.c, a tool like gem5 is more useful as it allows to keep track of global indicators such as execution time total number of ticks or cache miss rate and allows to track that second kind of side channels.

3. An ISA designer has two competing goals : make at a same time a useful and secure ISA. To be useful, the ISA can be CISC, allowing for optimizations in detail, to take advantage of all the hardware. It has to offer these capabilities to attract chip designers and make them want to use their products. But giving a lot of control to the user has the downside of giving a back door to sensitive parts, giving a direct access to some side channels, like with cflush allowing Flush+Reload attacks for example. So a compromise has to be found between giving control to the user and restricting some areas.

Part 5: Processor Optimization

1. I deepened my knowledge on side channels, and using it on the simple example of leaky-prog.c allowed to show how they work on principle. I understood some ways of hiding the information leakage with ZKT and the performance reduction that it causes.
2. For the first part, we don't have access to the variation in execution time of the processor (the simulation always give the same result for not secret and super secret). So the lower and upper bounds can only be inferred by submitting guesses to gradescope, instead of observing the processor.
3. How physical side channels could be used like energy consumption or heat? Is it realistic?